



**BANGLADESH UNIVERSITY OF
ENGINEERING AND TECHNOLOGY**

CSE 306

Computer Architecture Sessional

Assignment-1: 4-bit ALU Simulation

Section - B1

Group - 03

Members of the Group:

1. 2005076 – S. M. Kausar Parvej
2. 2005071 – Sagor Chanda
3. 2005069 – Souvik Mandol
4. 2005068 – Md. Sumon Hossain
5. 1705083 – Hozifa Rahman Hamim

Introduction:

An arithmetic logic unit (ALU) is a combinational digital circuit used to perform arithmetic and logic operations. It performs bitwise operations on integer binary numbers. It represents the fundamental building block of the central processing unit (CPU) of a computer. Modern CPUs contain very powerful and complex ALUs. In addition to ALUs, modern CPUs contain a control unit (CU).

The purpose of ALU is to perform mathematical operations such as addition, subtraction, multiplication and division. ALU comprises two main units. Arithmetic Unit and Logic Unit.

The arithmetic operations are like addition, subtraction, increment, decrement, transfer and logical operations are like NOT, OR, XOR, AND etc.

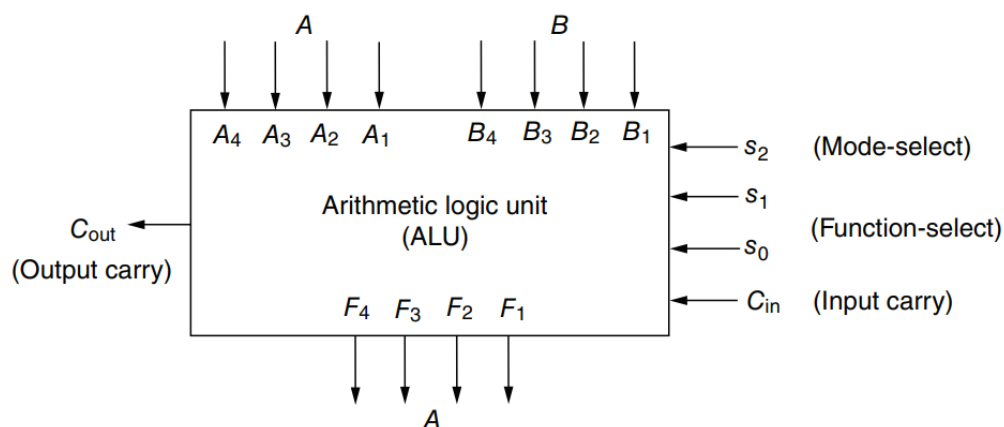


Figure 1: Block diagram of a 4-bit ALU

The arithmetic portion of the ALU is powered by a parallel adder. Achieving predicted outcomes for various arithmetic operations is made possible by using decoder to modify inputs to the IC. Furthermore, each integrated circuit (IC) has the capability to carry out logical operations. Additionally, certain efficient designs employ an IC for an operation other than the one for which it was designed.

In our ALU design, we have four status outputs, or flags. Carry Flag (C), Zero Flag (Z), Overflow Flag (V), and Sign Flag (S) are the symbols used to represent them.

- **CF:** Bit C is set 1 when the output carry of the ALU is 1, otherwise it is set 0.
- **SF:** Bit S is set 1 when the highest order bit of the output of the ALU is 1.
- **OF:** Bit V is set 1 if the X-OR of carries C3 and Cout is 1, otherwise it is set 0.
- **ZF:** Bit Z is set 1 if the result is zero, otherwise the Z bit is set 0.

Problem Specification with assigned instructions:

Design a 4-bit ALU equipped with a 4-bit flag output with three selection bits cs0, cs1, cs2. The ALU is designed to handle two 4-bits input numbers A, B

Control Signals			Functions	Modified Inputs		
CS2	CS1	CS0		X_i	Y_i	Z_0
0	0	0	Decrement A	A_i	1	0
0	0	1	Neg A	A_i'	0	1
x	1	0	OR	$A_i \text{ OR } B_i$	0	0
0	1	1	Subtract with borrow	A_i	B_i'	0
1	0	x	Subtract	A_i	B_i'	1
1	1	1	Complement A	A_i'	0	0

ALU also implements the following four flags which will be affected as per the rules of Assembly Language.

Flag Code	Name	Description
CF	Carry	Indicates an unsigned overflow scenario
ZF	Zero	When the result is zero then it set to 1
VF	Overflow	When sign overflow condition happens
SF	Sign	Represents the sign of the result

Table 1&2: Problem Specification

Detailed Design Steps with K-maps:

Design Step:

1. With the aid of a Decoder and a 4-bit full Adder, the arithmetic unit computes four arithmetic operations (Decrement A, Neg A, Subtract, Subtract with borrow) and two logical operations (OR and Complement A).
2. First, using the control selection bit (cs2 cs1 cs0) as the input of the 3X8 decoder the 4-bit input A, B are modified into X and Y respectively
3. Then X and Y are fed into the 4-bit parallel adder as the input.
4. Also, for the carry input (Cin) of the adder Zo is generated using decoder and some basic logic gates.
5. We had to ensure that all the internal carry generated in the adder must remain zero during the two logical operations. We modified our design according to that cause.
6. To keep the number of IC minimum we altered some logical function, such as: some consecutive not & and operations were performed via a single NOR gate. We also used left of NAND & XOR gate in the IC as to perform not operation.
7. The carry flag CF and sign flag SF directly came from the output of the adder (Cout and S3).
8. The overflow flag VF was calculated using input A3, B3 and output S3, Cout. ($VF = Cout \oplus C3$ and $C3 = S3 \oplus A3 \oplus B3$)
9. Lastly, the zero flag ZF was computed via the OR operation of the 4 output and then inverting the result. ($ZF = \sim (S3 + S2 + S1 + S0)$)

K-Maps:**For Xi:**

CS0 \ CS2 CS1	0	1
00	Ai	Ai'
01	Ai + Bi	Ai
11	Ai + Bi	Ai'
10	Ai	Ai

Using decoders with active low output:

$$\begin{aligned}
 X_i &= A_i (D_0 + D_3 + D_4 + D_5) + (A_i + B_i) (D_2 + D_6) + A_i' (D_1 + D_7) \\
 &= A_i (D_0 + D_2 + D_3 + D_4 + D_5 + D_6) + A_i (D_1 + D_7) + B_i (D_2 + D_6) \\
 &= A_i (D_1' D_7') + A_i' (D_1' D_7')' + B_i (D_2' D_6')' \\
 &= A_i \oplus (D_1' D_7')' + B_i (D_2' D_6')'
 \end{aligned}$$

For Yi:

CS0 \ CS2 CS1	0	1
00	1	0
01	0	Bi'
11	0	0
10	Bi'	Bi'

$$\begin{aligned}
 Y_i &= D_0 + B_i (D_3 + D_4 + D_5) \\
 &= (D_0')' + B_i (D_3' D_4' D_6')' \\
 &= (D_0')' + (B_i + (D_3' D_4' D_6'))'
 \end{aligned}$$

For Zi

CS0 CS2 CS1	0	1
00	0	1
01	0	0
11	0	0
10	1	1

$$Z_0 = D_1 + D_4 + D_5$$

$$= (D_1' D_4' D_5')'$$

Truth-Table:

Control Signals			Functions	Modified Inputs		
CS2	CS1	CS0		X_i	Y_i	Z_0
0	0	0	Decrement A	A_i	1	0
0	0	1	Neg A	A_i'	0	1
0	1	0	OR	$A_i \text{ OR } B_i$	0	0
1	1	0	OR	$A_i \text{ OR } B_i$	0	0
0	1	1	Subtract with borrow	A_i	B_i'	0
1	0	0	Subtract	A_i	B_i'	1
1	0	1	Subtract	A_i	B_i'	1
1	1	1	Complement A	A_i'	0	0

Block Diagram:

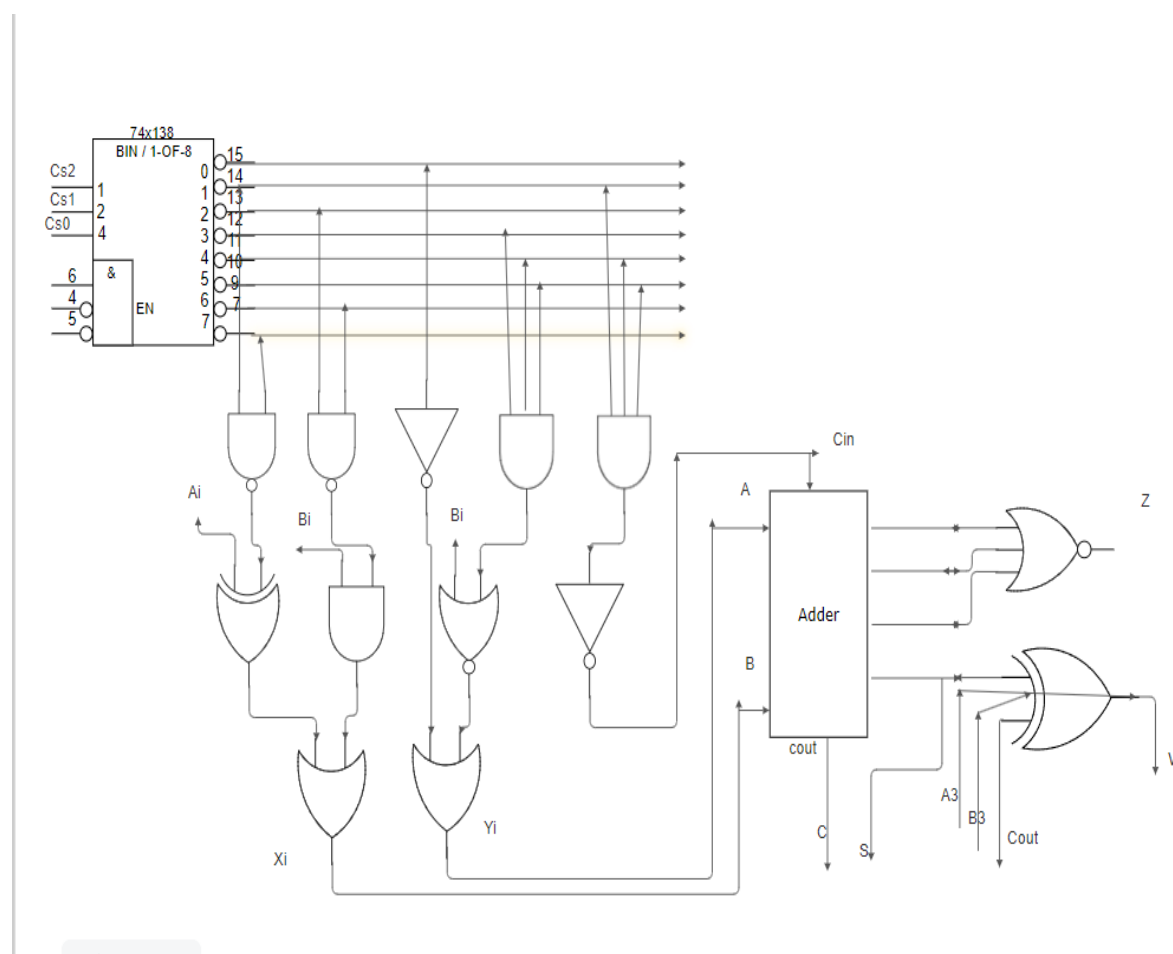


Figure 3: Block Diagram

Complete Circuit Diagram:

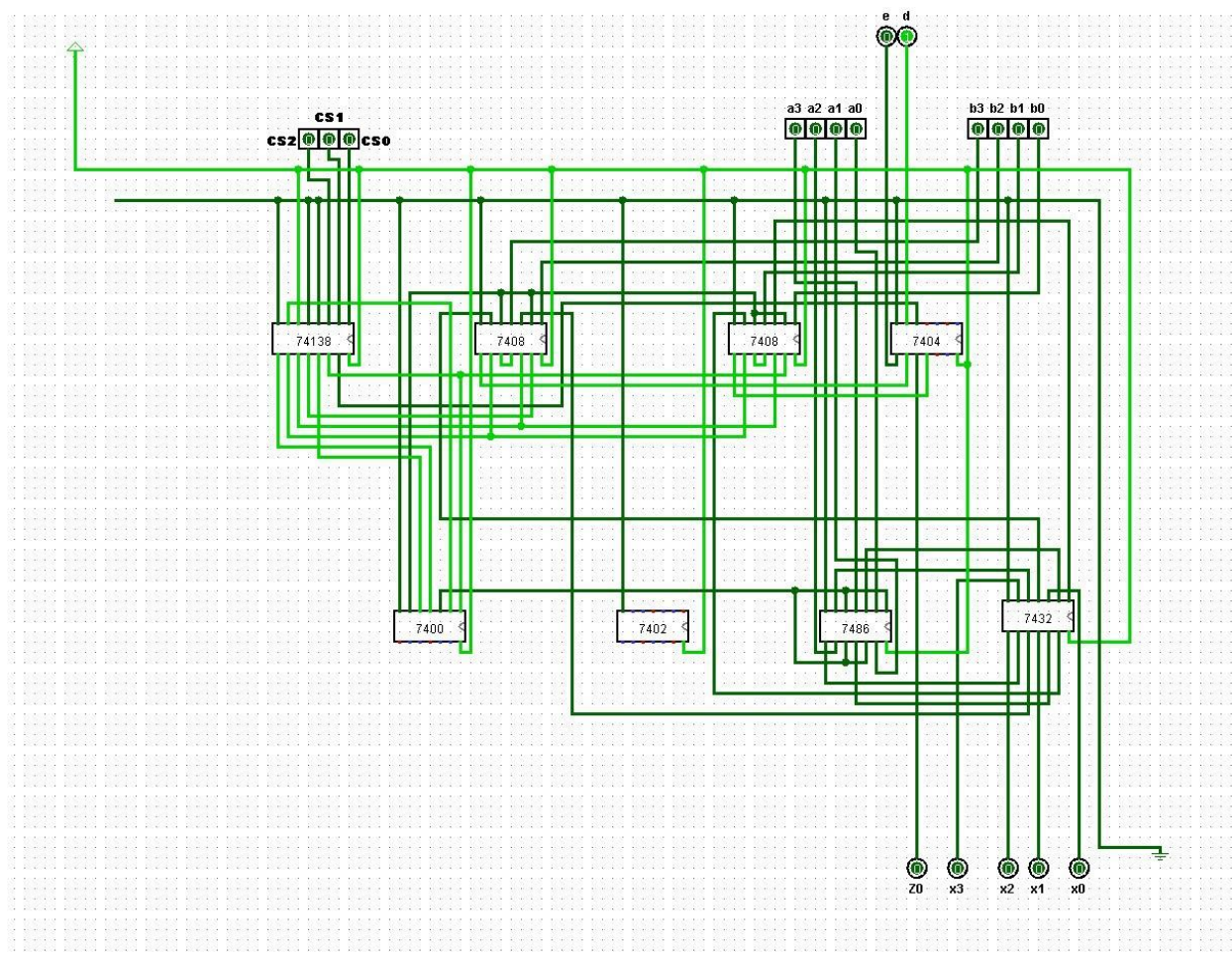
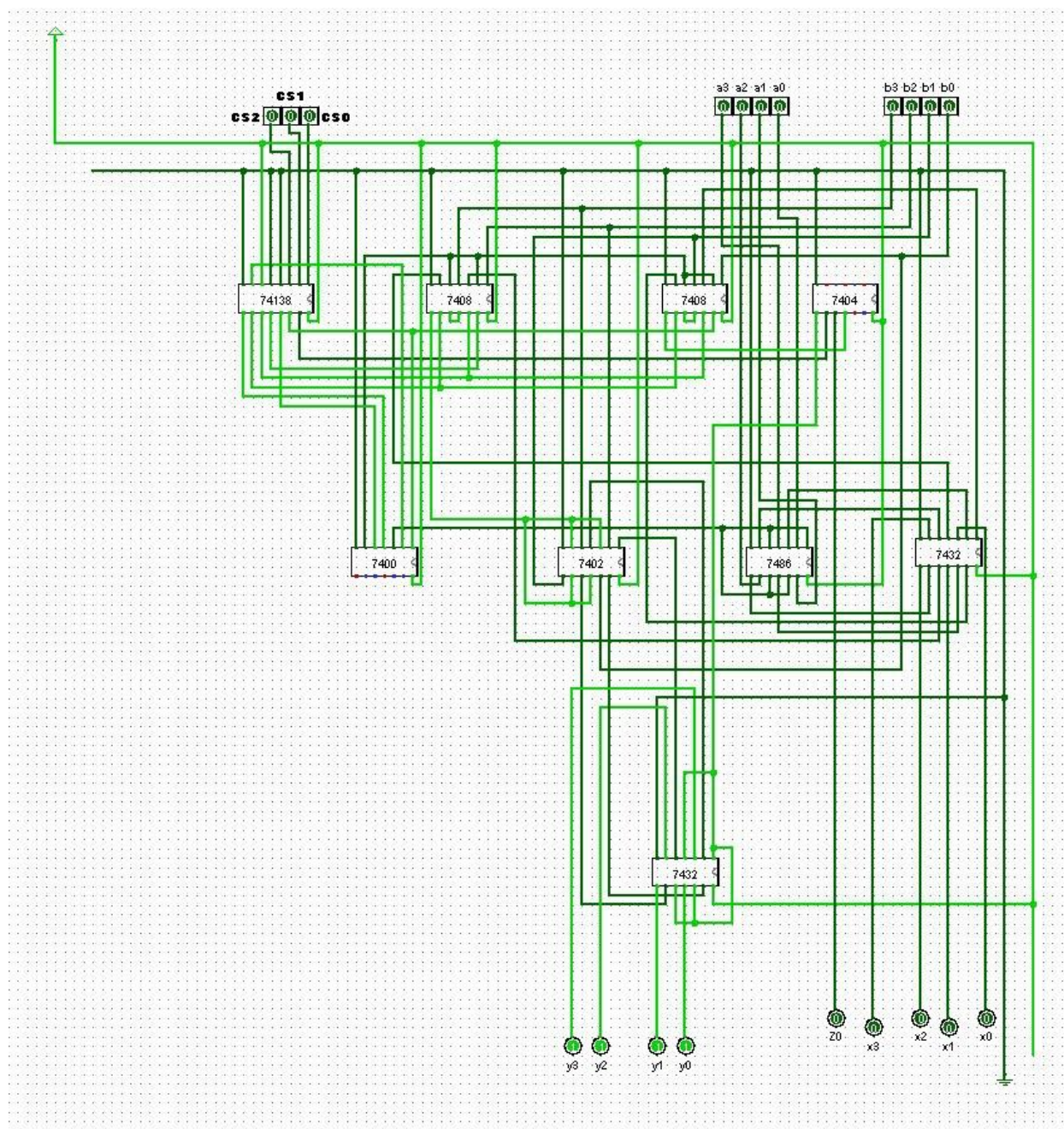


Figure 4: Finding modified A_i (X_i)

Figure 5: Finding modified B_i (Y_i)

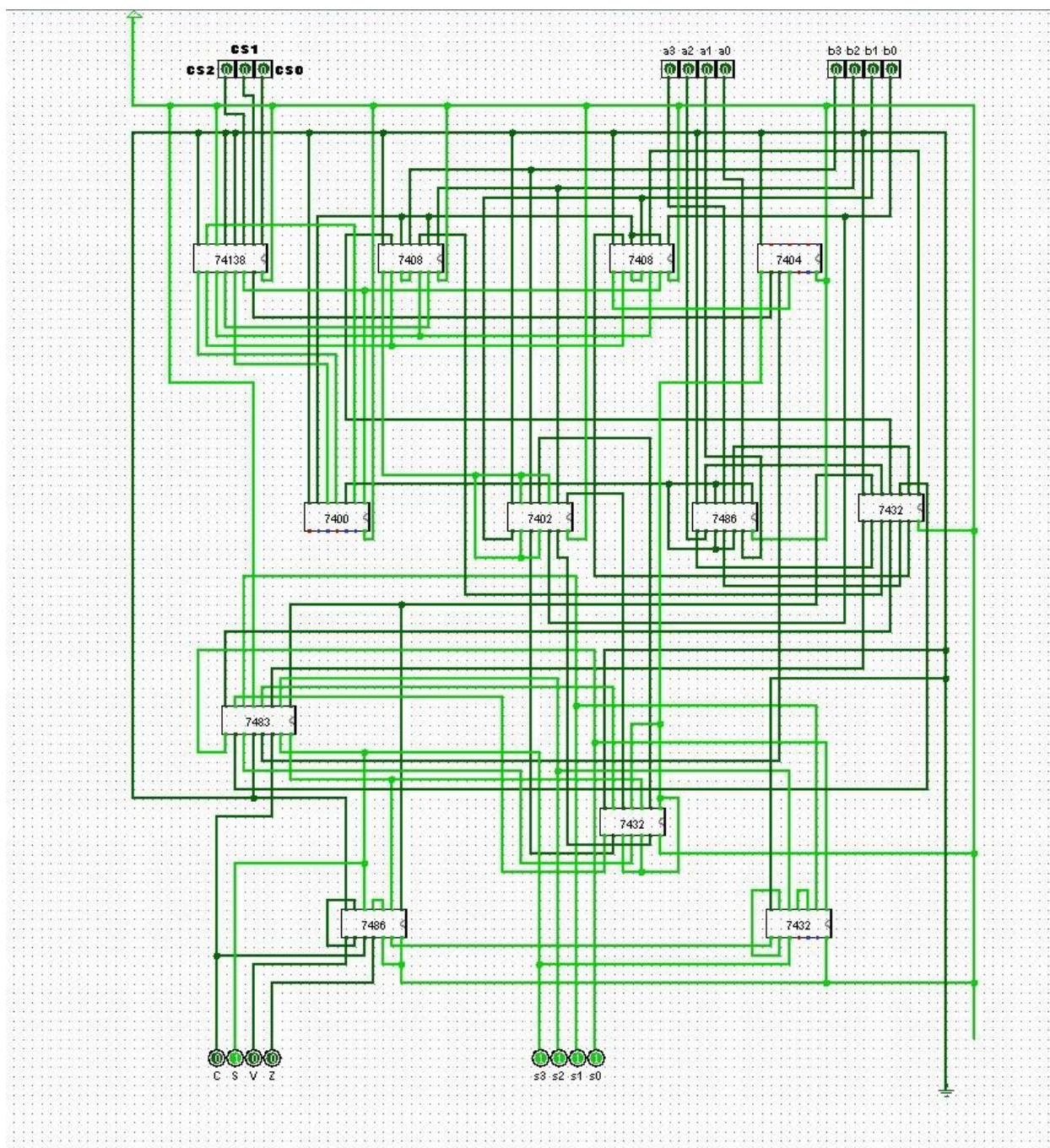


Figure 6: Full Diagram (unoptimized)

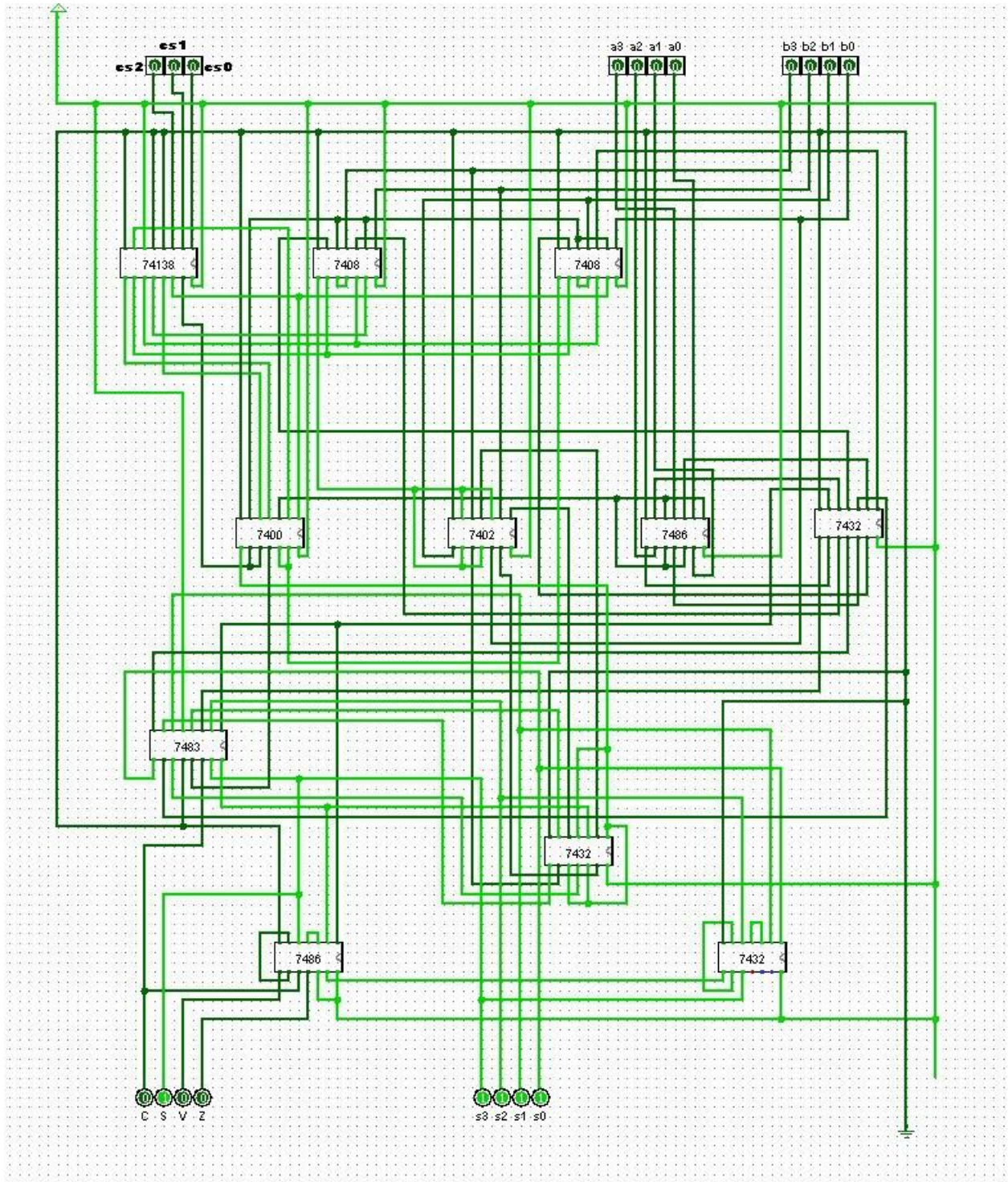


Figure 7: Full Diagram (optimized)

ICs used with count as a chart:

IC with Code	Quantity
IC 74138	1
IC 7408	2
IC 7400	1
IC 7402	1
IC 7486	2
IC 7483	1
IC 7432	3
TOTAL	11

Table: ICs used with quantity

The Simulator Used along with the Version Number:

Logisim - 2.7.1

Discussions:

In this assignment we were asked to implement a 4-bit ALU which performs 6 operations with the minimum numbers of ICs. Four arithmetic operations and two logical operations.

We had to work very hard under strict examination to get the design with the fewest number of integrated circuits. The logical NOT operation using IC7486 (XOR) & IC7400(NAND) and the manipulation the IC7483 (Adder) inputs to do different arithmetic & logical operations were among the optimizations we had to make in order to accomplish this. Everybody in the group chipped in with their own design ideas and we tried all of them in coming to the final result.

During the hardware implementation we faced different issues. Planning the arrangement of various components was one of them. The wire was getting mostly tangled up together and everything was getting really messy. We had to connect the wires in a way that minimized the amount of length that they crossed between the connections in order to maintain a clean hardware design. To avoid damaging the IC or other components, care was taken when connecting the power and ground. Also, resistance was added to the LED and Switch to protect them.

Contribution of Each Member:

1. 2005076 – S. M. Kausar Parvej
 - Finding equation of Xi, Yi and Zo
 - Circuit implementation in Logisim
 - Hardware implementation
2. 2005071 – Sagor Chanda
 - Discussion and coming up with ideas
 - Circuit implementation in Logisim
 - Hardware implementation
3. 2005069 – Souvik Mandol
 - Circuit implementation in Logisim
 - Participation in Group Discussion
 - Hardware implementation
4. 2005068 – Md. Sumon Hossain
 - Participation in Group Discussion
 - Lab Report writing
 - Hardware implementation
5. 1705083 – Hozifa Rahman Hamim
 - Participation in Group Discussion
 - Lab Report writing